

MODIFYING A LIST OR ARRAY IN PLACE

Python doesn't pass a copy of lists, and JavaScript doesn't pass a copy of arrays, to function calls. Rather, they pass a reference to the list. Therefore, any changes made to a list or array (such as the ones in `maze` and `visited`) remain even after the function returns. This is called modifying the list *in place*. For recursive functions, you can think of the maze data structure and collection of visited coordinates as a single copy shared among all the recursive function calls, unlike the `x` and `y` arguments. This is why the data structure in `MAZE` is still modified after the first call to `solveMaze()` returns.

To get a better idea of how this algorithm works, uncomment the two `printMaze(MAZE)` calls ❸ ❹ inside the `solveMaze()` function. These will display the maze data structure as it attempts new paths, reaches dead ends, backtracks, and tries different paths.